

算法设计与分析期末实验报告

Kruskal 与 Prim 最小生成树算法

专业 智能科学与技术 姓名 史峰源 学号 2412526

摘要：本实验针对一个含 6 个顶点和 10 条边的无向带权连通图，分别使用 Kruskal 算法和从 V_1 出发的 Prim 算法求解最小生成树。程序采用 C++ 实现：Kruskal 使用并查集判环，Prim 使用邻接表和最小优先队列维护当前割上的候选边。两种算法均得到 5 条边，总权值为

$$1 + 2 + 3 + 4 + 5 = 15.$$

实验进一步利用割性质证明该图的最小生成树唯一，并分析两种贪心策略在选边范围、数据结构、复杂度及结果表现方面的区别。

关键词：最小生成树；Kruskal 算法；Prim 算法；贪心算法；并查集；优先队列

1 实验目的

1. 理解最小生成树问题的数学定义以及贪心算法能够得到最优解的依据；
2. 掌握 Kruskal 算法的全局边排序策略和并查集判环方法；
3. 掌握 Prim 算法从一个起点逐步扩展生成树的方法，并理解最小优先队列的作用；
4. 使用 C++ 独立实现两种算法，对输出结果进行合法性和一致性验证；
5. 通过过程图、结果表和复杂度分析，比较两种算法的共同点、区别与适用场景。

2 问题描述与图模型

2.1 实验问题

题目给出如图 1 所示的无向带权连通图 $G = (V, E)$ 。顶点集合为

$$V = \{V_1, V_2, V_3, V_4, V_5, V_6\},$$

边集合及权值见表 1。实验要求分别使用 Kruskal 和 Prim 算法求出该图的最小生成树，并给出程序、运行结果及分析。

实验图：无向带权连通图

顶点数 $|V| = 6$, 边数 $|E| = 10$

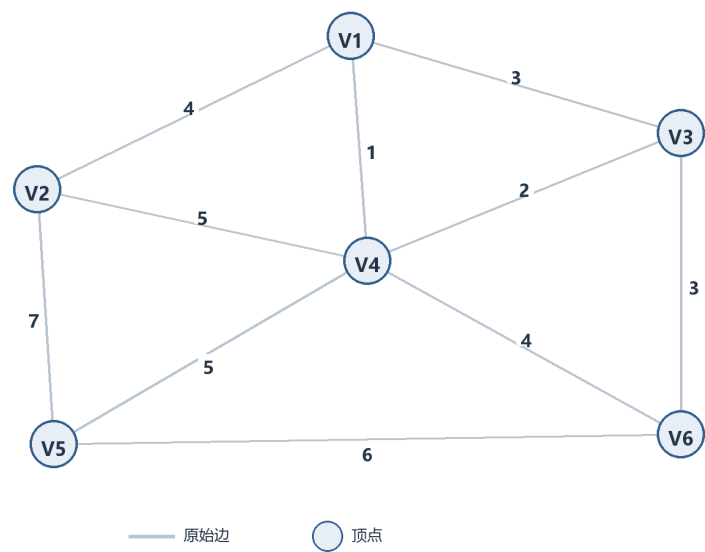


图 1: 题目给出的无向带权连通图 (Python 重绘)

表 1: 原图的边与权值										
边	V_1V_2	V_1V_3	V_1V_4	V_2V_4	V_2V_5	V_3V_4	V_3V_6	V_4V_5	V_4V_6	V_5V_6
权值	4	3	1	5	7	2	3	5	4	6

2.2 最小生成树定义

若子图 $T = (V, E_T)$ 满足以下条件, 则称 T 为图 G 的一棵生成树:

- 1. 包含原图的全部顶点;
- 2. 连通且不含环;
- 3. 对 $|V|=n$ 个顶点, 恰好含有 $n-1$ 条边。

在所有生成树中, 使

$$w(T) = \sum_{e \in E_T} w(e)$$

最小的生成树称为最小生成树 (Minimum Spanning Tree, MST)。本题 $|V|=6$, 因此任何生成树都必须恰好包含 5 条边。

3 算法基本原理

3.1 贪心选择的理论依据

割性质: 将顶点集合划分为两个非空集合 S 与 $V-S$, 所有一端位于 S 、另一端位于 $V-S$ 的边称为该割的跨边。若某条边是该割上的唯一最轻边, 则它必定属于每一棵最小生成树。Kruskal 和 Prim 都在反复选择某个安全的最轻边, 只是两者考察的割不同。

环性质：若一个环中存在严格最重的边，则该边不可能属于任何最小生成树。Kruskal 在加入一条边会形成环时舍弃该边；Prim 则只允许候选边连接“已访问集合”和“未访问集合”，因此每次加入一个新顶点，天然避免成环。

3.2 Kruskal 算法

Kruskal 将所有边按权值从小到大排序，并依次检查。若当前边连接两个不同连通分量，则将其加入生成树；若两端已经连通，加入它会形成环，因此舍弃。程序使用并查集维护各顶点所属的连通分量。

为了让同权边的处理顺序完全可复现，本实验采用确定性排序键

$$(w, \min(u, v), \max(u, v)),$$

即先比较权值，再比较较小端点，最后比较大端点。

算法 1 Kruskal 最小生成树算法

Require: 连通无向带权图 $G = (V, E)$

Ensure: 最小生成树边集 T

```

1: 将  $E$  按  $(w, \min(u, v), \max(u, v))$  升序排序
2: 初始化并查集，使每个顶点单独构成一个集合
3:  $T \leftarrow \emptyset$ 
4: for 每条边  $(u, v, w)$  do
5:   if  $u$  与  $v$  属于不同集合 then
6:      $T \leftarrow T \cup \{(u, v, w)\}$ 
7:     合并  $u$  与  $v$  所在集合
8:   if  $|T| = |V| - 1$  then
9:     break
10: return  $T$ 

```

3.3 Prim 算法

Prim 从一个指定顶点开始，维护已进入生成树的顶点集合 S 。每轮从所有连接 S 与 $V - S$ 的跨边中选择权值最小者，将其另一端的新顶点加入 S ，直到所有顶点均被覆盖。本实验固定从 V_1 开始，并使用最小优先队列保存候选边。

算法 2 Prim 最小生成树算法

Require: 连通无向带权图 $G = (V, E)$ ，起点 $s = V_1$

Ensure: 最小生成树边集 T

```

1:  $S \leftarrow \{s\}$ ,  $T \leftarrow \emptyset$ 
2: 将  $s$  的所有邻接边压入最小优先队列
3: while 优先队列非空且  $|T| < |V| - 1$  do
4:   取出排序键最小的候选边  $(u, v, w)$ 
5:   if 该边指向的顶点已经访问 then
6:     continue
7:   将  $(u, v, w)$  加入  $T$ ，并将新顶点加入  $S$ 
8:   将新顶点通向未访问顶点的边压入优先队列
9: return  $T$ 

```

3.4 两种算法的主要区别

表 2: Kruskal 与 Prim 的比较		
比较项目	Kruskal	Prim
贪心视角	面向全图的边，每次选不成环的全局最轻边	面向当前生成树，每次选当前割上的最轻跨边
中间状态	多个连通分量逐步合并	始终保持一棵从起点向外生长的树
核心结构	边数组、排序、并查集	邻接表、访问标记、最小优先队列
起点影响	不需要起点	需要起点；起点可能改变选边过程，非唯一时也可能改变最终边集
典型复杂度	$O(E \log E)$	邻接表与二叉堆实现为 $O(E \log E)$ ，也常写作 $O(E \log V)$
较适合场景	边较少的稀疏图，或输入天然以边集形式给出	邻接结构明显的图；稠密图还可利用邻接矩阵实现为 $O(V^2)$

4 程序设计与代码解释

4.1 整体程序结构

程序而是按功能拆分为若干独立模块。既便于分别检查 Kruskal 和 Prim，也能让两者共用边结构、权值统计和结果验证函数。

表 3: C++ 程序模块及其功能	
模块或函数	作用
Edge	保存一条边的起点、终点和权值
DisjointSet	维护连通分量，为 Kruskal 和结果复检提供判环功能
normalizedKey	规范化无向边，并规定同权边的确定性顺序
kruskal	对全部边排序，选择不会形成环的边
prim	从指定起点出发，用最小优先队列选择当前最轻跨边
totalWeight	对生成树中的边权求和
isSpanningTree	检查边数、无环性和连通性
normalizedEdgeSet	忽略无向边书写方向后比较两个结果的边集
printResult	输出选边次序、边数和总权值
main	构造题目数据，调用两种算法，并汇总验证结果

程序的数据流可以概括为

输入边集 $\longrightarrow \left\{ \begin{array}{l} \text{Kruskal: 排序 + 并查集判环,} \\ \text{Prim: 邻接表 + 最小优先队列} \end{array} \right. \longrightarrow \text{输出边集} \longrightarrow \text{合法性与权值验证.}$

4.2 边结构与同权边排序

代码清单 1: 边结构和规范化排序键

```

1 struct Edge {
2     int u;        // 一个端点
3     int v;        // 另一个端点
4     int weight;   // 边权
5 };
6
7 tuple<int, int, int> normalizedKey(const Edge& edge) {
8     return {edge.weight,
9             min(edge.u, edge.v),
10            max(edge.u, edge.v)};
11 }
12
13 bool edgeLess(const Edge& a, const Edge& b) {
14     return normalizedKey(a) < normalizedKey(b);
15 }

```

Edge 中的 u 、 v 是两个端点， $weight$ 是边权。由于本题为无向图，程序不能把 $V_3-V_4(2)$ 和 $V_4-V_3(2)$ 当作两种不同结果。

$normalizedKey$ 返回三元组

$(weight, \min(u, v), \max(u, v))$ 。

C++ 的 $tuple$ 按字段从左到右进行字典序比较，因此代码首先比较权值；权值相同时比较较小端点；若仍相同，再比较较大端点。例如权值同为 3 时， $V_1-V_3(3)$ 的键为 $(3, 1, 3)$ ， $V_3-V_6(3)$ 的键为 $(3, 3, 6)$ ，所以前者先被 Kruskal 检查。这一规则不影响最小权值，只负责使每次运行的过程一致。

4.3 并查集：如何判断加入一条边是否成环

代码清单 2: 并查集的路径压缩与按秩合并

```

1 class DisjointSet {
2 public:
3     explicit DisjointSet(int n)
4         : parent(n + 1), rank_value(n + 1, 0) {
5         iota(parent.begin(), parent.end(), 0);
6     }
7
8     int find(int x) {
9         if (parent[x] != x) {
10             parent[x] = find(parent[x]);
11         }
12         return parent[x];
13     }
14
15     bool unite(int a, int b) {
16         int root_a = find(a);
17         int root_b = find(b);

```

```

18     if (root_a == root_b) {
19         return false;
20     }
21
22     if (rank_value[root_a] < rank_value[root_b]) {
23         swap(root_a, root_b);
24     }
25     parent[root_b] = root_a;
26     if (rank_value[root_a] == rank_value[root_b]) {
27         ++rank_value[root_a];
28     }
29     return true;
30 }
31
32 private:
33     vector<int> parent;
34     vector<int> rank_value;
35 };

```

这段代码的含义可分为四步：

1. 构造函数创建长度为 $n + 1$ 的数组，因为本题顶点编号从 1 开始；`iota` 令初始状态满足 `parent[i]=i`，即每个顶点各自成组。
2. `find(x)` 沿父结点寻找代表元。语句 `parent[x] = find(parent[x])` 在返回时将路径上的结点直接连到根，这就是路径压缩。
3. `unite(a,b)` 先求两个根。如果 `root_a == root_b`，说明 a 和 b 已经存在路径相连，此时再加入边 (a,b) 必然形成环，所以返回 `false`。
4. 若两个根不同，则按秩把较矮的树接到较高的树下，并返回 `true`，表示该边可以安全加入。

以 Kruskal 的第三次检查为例：接受 $V_1-V_4(1)$ 和 $V_3-V_4(2)$ 后， V_1, V_3, V_4 已属于同一集合。检查 $V_1-V_3(3)$ 时，`find(1)` 与 `find(3)` 返回同一代表元，于是 `unite(1,3)` 返回 `false`，程序舍弃该边。

4.4 Kruskal 核心代码与语句对应关系

代码清单 3: Kruskal 算法核心实现

```

1  vector<Edge> kruskal(int vertex_count, vector<Edge> edges) {
2      sort(edges.begin(), edges.end(), edgeLess);
3
4      DisjointSet dsu(vertex_count);
5      vector<Edge> mst;
6
7      for (const Edge& edge : edges) {
8          if (dsu.unite(edge.u, edge.v)) {
9              mst.push_back(edge);
10             if (static_cast<int>(mst.size())
11                 == vertex_count - 1) {
12                 break;
13             }
14         }

```

```

15     }
16
17     if (static_cast<int>(mst.size()) != vertex_count - 1) {
18         throw runtime_error(
19             "The graph is disconnected; no spanning tree exists.");
20     }
21     return mst;
22 }

```

1. `sort(..., edgeLess)` 对所有边执行一次全局排序，对应算法中的“从最轻边开始依次检查”。
2. `DisjointSet dsu(vertex_count)` 初始化 6 个独立连通分量；`mst` 用来保存最终被接受的边。
3. 循环中的 `dsu.unite(edge.u, edge.v)` 同时完成两个动作：判断两端是否已连通；若未连通，则立即合并两个分量。
4. 只有 `unite` 返回 `true` 时才执行 `mst.push_back(edge)`，因此 `mst` 中始终不会出现环。
5. 当 `mst.size()==vertex_count-1` 时已经选出 5 条边。对 6 个顶点而言，这时无需继续检查其余较重边。
6. 若循环结束仍不足 5 条边，说明输入图不连通，不存在覆盖全部顶点的生成树，因此程序抛出异常，而不是返回一个不完整结果。

本题排序后的开头依次为

$$V_1-V_4(1), \quad V_3-V_4(2), \quad V_1-V_3(3), \quad V_3-V_6(3), \quad V_1-V_2(4), \quad V_4-V_6(4), \dots$$

前两条边合并成功；第三条边因成环而失败；第四条边继续合并新顶点 V_6 。因此代码行为与??中的接受、舍弃过程一一对应。

4.5 Prim 核心代码：候选边如何进入最小堆

代码清单 4: 建立邻接表、候选边和访问集合

```

1  vector<vector<pair<int, int>>> graph(vertex_count + 1);
2  for (const Edge& edge : edges) {
3      graph[edge.u].push_back({edge.v, edge.weight});
4      graph[edge.v].push_back({edge.u, edge.weight});
5  }
6
7  // weight, smaller endpoint, larger endpoint, from, to
8  using Candidate = tuple<int, int, int, int, int>;
9  priority_queue<Candidate,
10                 vector<Candidate>,
11                 greater<Candidate>> candidates;
12  vector<bool> visited(vertex_count + 1, false);
13  vector<Edge> mst;
14
15  auto addCandidates = [&](int vertex) {
16      visited[vertex] = true;
17      for (const auto& [next, weight] : graph[vertex]) {
18          if (!visited[next]) {
19              candidates.push(
20                  {weight,
21                     min(vertex, next),

```

```

22         max(vertex, next),
23         vertex,
24         next});
25     }
26 }
27 };

```

一条无向边需要同时加入两个端点的邻接表，因此代码对每条边执行两次 `push_back`。例如 $V_1 - V_4(1)$ 既出现在 V_1 的邻接表中，也出现在 V_4 的邻接表中。

`Candidate` 的五个字段依次是：

(权值, 较小端点, 较大端点, from, to)。

前三个字段决定优先级，后两个字段记录本次扩展方向。`priority_queue` 默认是最大堆；这里使用 `greater` 作为比较器，将其改为最小堆，所以 `top()` 始终给出当前排序键最小的候选边。

`addCandidates(vertex)` 完成两项工作：先把新顶点标记为已访问，再把该顶点通向所有未访问邻点的边压入最小堆。它对应 Prim 中“将新顶点并入当前树，并更新割上的候选边”这一步。

代码清单 5: Prim 从最小堆取边并扩展生成树

```

1  addCandidates(start);
2  while (!candidates.empty() &&
3         static_cast<int>(mst.size()) < vertex_count - 1) {
4      auto [weight, smaller, larger, from, to]
5          = candidates.top();
6      candidates.pop();
7      (void)smaller;
8      (void)larger;
9
10     if (visited[to]) {
11         continue;
12     }
13
14     mst.push_back({from, to, weight});
15     addCandidates(to);
16 }
17
18 if (static_cast<int>(mst.size()) != vertex_count - 1) {
19     throw runtime_error(
20         "The graph is disconnected; no spanning tree exists.");
21 }

```

从 V_1 开始时，`addCandidates(1)` 首先放入 $V_1 - V_4(1)$ 、 $V_1 - V_3(3)$ 和 $V_1 - V_2(4)$ ，堆顶显然是 $V_1 - V_4(1)$ 。程序把它加入 `mst` 后调用 `addCandidates(4)`，新加入的候选边中包含权值为 2 的 $V_4 - V_3(2)$ ，因此第二轮选择该边。

候选边进入堆以后不会被主动删除。若另一条更轻边已经使它的 `to` 端进入生成树，则弹出时 `visited[to]` 为真，程序直接 `continue`。例如 V_3 通过 $V_4 - V_3(2)$ 加入后，早先放入堆中的 $V_1 - V_3(3)$ 就成为失效候选边。这种“弹出时再跳过”的写法称为惰性删除，避免了在优先队列内部查找并删除任意元素。

4.6 结果统计、合法性检查与主函数

代码清单 6: 检查算法输出是否为合法生成树

```
1 bool isSpanningTree(int vertex_count,
2                     const vector<Edge>& edges) {
3     if (static_cast<int>(edges.size())
4         != vertex_count - 1) {
5         return false;
6     }
7
8     DisjointSet dsu(vertex_count);
9     for (const Edge& edge : edges) {
10         if (!dsu.unite(edge.u, edge.v)) {
11             return false;
12         }
13     }
14
15     const int root = dsu.find(1);
16     for (int vertex = 2; vertex <= vertex_count; ++vertex) {
17         if (dsu.find(vertex) != root) {
18             return false;
19         }
20     }
21     return true;
22 }
```

验证函数依次检查三件事:

1. 边数必须恰好是 $V - 1$;
2. 再次使用并查集逐边合并, 若某次合并失败, 说明结果中存在环;
3. 最后检查所有顶点的代表元是否与 V_1 相同, 确认整棵树连通。

三项同时成立才返回 `true`。

代码清单 7: 主函数中的算法调用与最终判定

```
1 const vector<Edge> kruskal_mst =
2     kruskal(vertex_count, edges);
3 const vector<Edge> prim_mst =
4     prim(vertex_count, edges, 1);
5
6 printResult("Kruskal algorithm", kruskal_mst);
7 printResult("Prim algorithm (start from V1)", prim_mst);
8
9 const bool same_weight =
10     totalWeight(kruskal_mst) == totalWeight(prim_mst);
11 const bool same_edges =
12     normalizedEdgeSet(kruskal_mst)
13     == normalizedEdgeSet(prim_mst);
14 const bool kruskal_valid =
15     isSpanningTree(vertex_count, kruskal_mst);
```

```

16  const bool prim_valid =
17      isSpanningTree(vertex_count, prim_mst);
18
19  return kruskal_valid && prim_valid && same_weight ? 0 : 1;

```

`prim(vertex_count, edges, 1)` 的最后一个参数明确指定从 V_1 开始。`same_weight` 检查两种算法所得总权值是否一致；`same_edges` 将端点方向规范化以后比较边集。

程序的返回条件只要求“两者均为合法生成树且总权值一致”，没有强制要求 `same_edges` 为真。这是因为一般图可能存在多棵边集不同、总权值相同的最小生成树。本题已经另外证明 MST 唯一，所以本次运行中 `same_edges` 也应当为真；若为假，应继续检查输入或实现。

4.7 代码语句与本题结果的对应

表 4: 关键代码如何产生本题输出

关键代码	算法含义	本题中的直接表现
<code>sort</code>	将边按权值和规范化端点排序	Kruskal 首先检查权值 1、2、3 的边
<code>unite</code>	判断两端是否属于不同连通分量	V_1V_3 、 V_4V_6 、 V_2V_4 因成环被舍弃
<code>addCandidates(1)</code>	从 Prim 起点建立第一批跨边	第一轮在权值 1、3、4 中选择 $V_1V_4(1)$
<code>top()</code>	取当前割上的最轻候选边	第二轮选择新出现的 $V_4V_3(2)$
<code>visited[to]</code>	跳过已经失效的旧候选边	$V_1V_3(3)$ 在 V_3 已访问后不再被选
<code>mst.size()==V-1</code>	已得到生成树所需的全部边	两种算法都在选满 5 条边后结束
<code>totalWeight</code>	累加所选边权	得到 $1 + 2 + 3 + 4 + 5 = 15$
<code>isSpanningTree</code>	复检边数、无环性和连通性	两项合法性输出均为 YES

5 实验过程与运行结果

5.1 Kruskal 逐边检查过程

按确定性排序规则，边的检查与处理结果见??。当已经选出 5 条边时，生成树完成，剩余的 $V_5 - V_6(6)$ 和 $V_2 - V_5(7)$ 无需继续检查。

表 5: Kruskal 算法的实际执行过程

检查次序	边	处理	原因
1	$V_1 - V_4(1)$	接受	当前最轻，连接两个独立分量
2	$V_3 - V_4(2)$	接受	V_3 与已有分量尚未连通
3	$V_1 - V_3(3)$	舍弃	$V_1 - V_4 - V_3$ 已连通，加入后成环
4	$V_3 - V_6(3)$	接受	引入新顶点 V_6
5	$V_1 - V_2(4)$	接受	引入新顶点 V_2
6	$V_4 - V_6(4)$	舍弃	$V_4 - V_3 - V_6$ 已连通，加入后成环
7	$V_2 - V_4(5)$	舍弃	$V_2 - V_1 - V_4$ 已连通，加入后成环
8	$V_4 - V_5(5)$	接受	引入 V_5 ，边数达到 5

Kruskal 算法逐边检查过程

排序规则：权值 → 较小端点 → 较大端点

此前已接受 当前接受 当前舍弃 (成环) 尚未处理

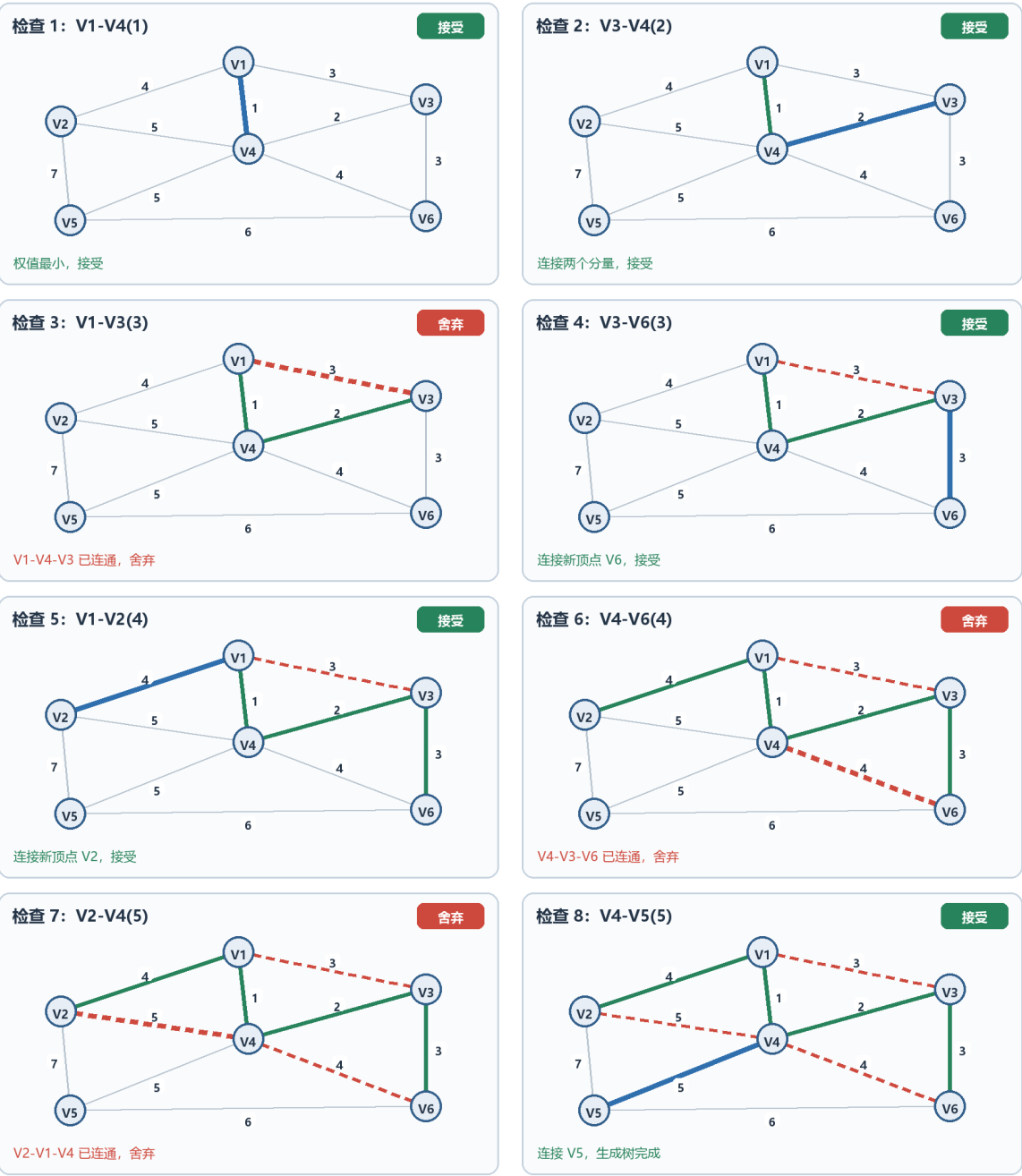


图 2: Kruskal 算法逐边接受与舍弃过程

5.2 Prim 从 V_1 出发的扩展过程

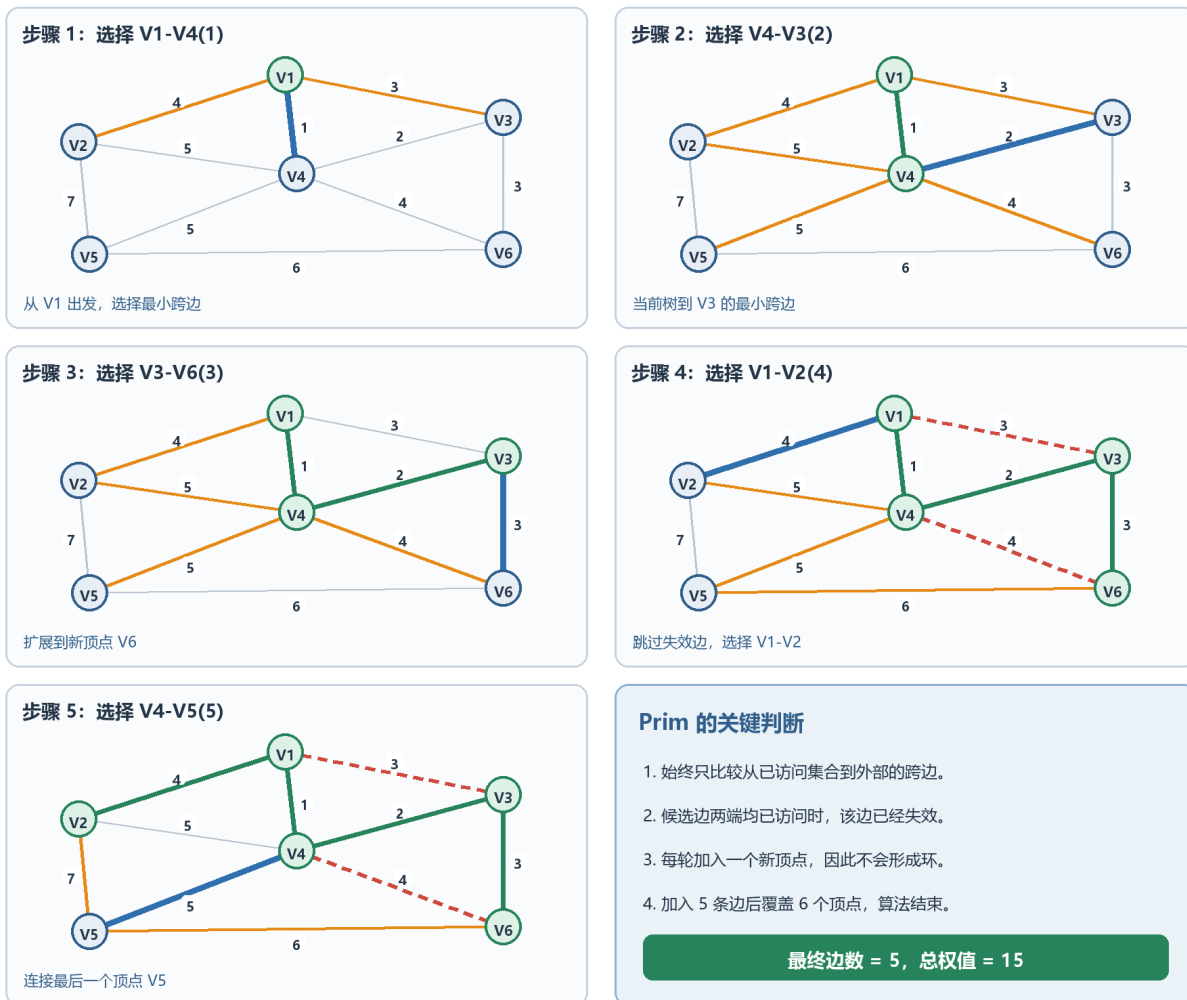
Prim 始终在当前访问集合 S 与外部顶点之间选择最轻跨边, 具体步骤见表 ??, 图形化过程见图 ??。例如第三步完成后 $S = \{V_1, V_3, V_4, V_6\}$, 原候选边 $V_1-V_3(3)$ 和 $V_4-V_6(4)$ 的两端都已经访问, 因此成为失效边; 下一条有效的最轻跨边是 $V_1-V_2(4)$ 。

表 6: Prim 算法从 V_1 出发的实际执行过程

步骤	选择的边	加入边后已访问顶点集合
1	$V_1 - V_4(1)$	$\{V_1, V_4\}$
2	$V_4 - V_3(2)$	$\{V_1, V_3, V_4\}$
3	$V_3 - V_6(3)$	$\{V_1, V_3, V_4, V_6\}$
4	$V_1 - V_2(4)$	$\{V_1, V_2, V_3, V_4, V_6\}$
5	$V_4 - V_5(5)$	$\{V_1, V_2, V_3, V_4, V_5, V_6\}$

Prim 算法从 V_1 出发的扩展过程

橙色为当前割上的候选边，蓝色为本轮选择，绿色为已加入生成树

图 3: Prim 算法从 V_1 出发的逐步扩展过程

5.3 程序实际输出

程序使用命令

```
g++ -std=c++17 -Wall -Wextra -pedantic solution.cpp -o solution.exe
```

编译，并执行 solution.exe。实际输出如下。

代码清单 8: C++ 程序实际运行输出

```

1  Kruskal algorithm
2  Step  Selected edge  Weight
3      1  V1-V4        1
4      2  V3-V4        2
5      3  V3-V6        3
6      4  V1-V2        4
7      5  V4-V5        5
8  Edge count: 5
9  Total weight: 15
10
11 Prim algorithm (start from V1)
12 Step  Selected edge  Weight
13      1  V1-V4        1
14      2  V4-V3        2
15      3  V3-V6        3
16      4  V1-V2        4
17      5  V4-V5        5
18 Edge count: 5
19 Total weight: 15
20
21 Verification
22 Kruskal is a spanning tree: YES
23 Prim is a spanning tree: YES
24 Same total weight: YES
25 Same normalized edge set: YES
26 Verification passed: YES

```

Kruskal 得到的选边顺序为

$$V_1-V_4(1), \quad V_3-V_4(2), \quad V_3-V_6(3), \quad V_1-V_2(4), \quad V_4-V_5(5).$$

Prim 得到的第二条边显示为 $V_4-V_3(2)$ ，这是因为该边在 Prim 中从当前树内的 V_4 指向树外的 V_3 ；它与 Kruskal 输出的 $V_3-V_4(2)$ 是同一条无向边。两种算法最终边集相同，总权值均为

$$w(T) = 1 + 2 + 3 + 4 + 5 = \boxed{15}.$$

两种算法的最终结果与选边顺序

圆形编号表示选边顺序；无向边 V_4-V_3 与 V_3-V_4 表示同一条边

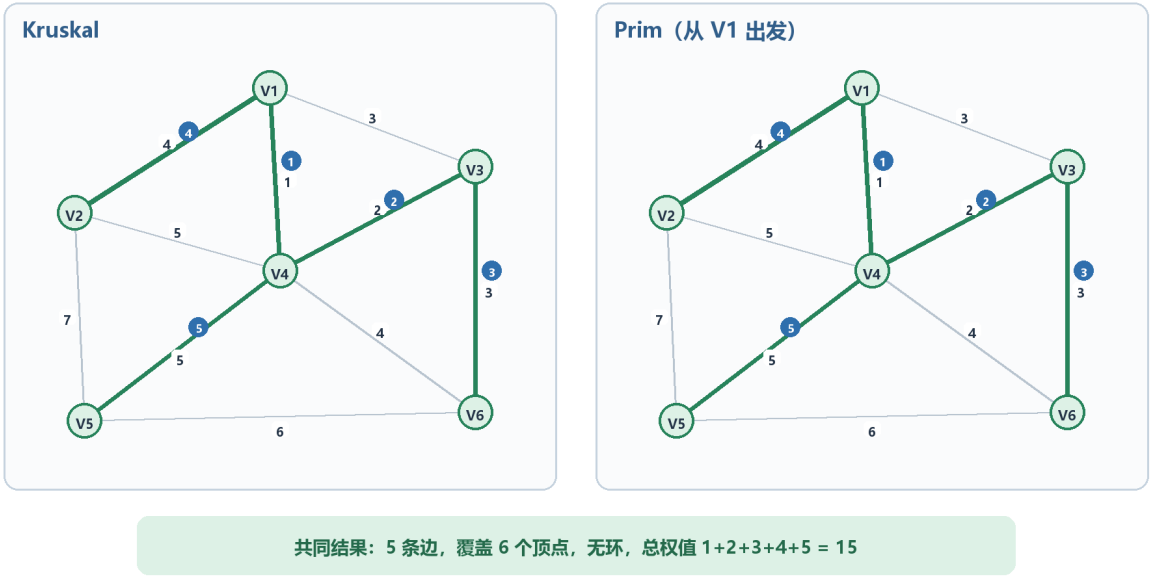


图 4: 两种算法的最终边集与选边顺序对比

6 结果分析

6.1 生成树合法性

输出包含 6 个顶点和 5 条边。由图可见所有顶点均连通；程序的并查集复检也表明不存在环。因此输出满足“覆盖全部顶点、连通、无环、边数为 $V - 1$ ”四项条件，确实是一棵生成树。

6.2 最优性与唯一性证明

仅仅看到两种算法给出相同结果，还不能单独构成最优性的数学证明。这里使用割性质进一步分析。对单顶点割 $\{V_i\}$ ，列出的边都是该割上的唯一最轻边，因此它们必须出现在每一棵最小生成树中。

表 7: 利用唯一最轻跨边证明 MST 唯一

割的一侧	唯一最轻跨边	权值
$\{V_1\}$	$V_1 - V_4$	1
$\{V_2\}$	$V_1 - V_2$	4
$\{V_3\}$	$V_3 - V_4$	2
$\{V_5\}$	$V_4 - V_5$	5
$\{V_6\}$	$V_3 - V_6$	3

这 5 条必须边互不相同，并且恰好构成一棵覆盖全部 6 个顶点的树，所以不存在替换其中任意一条边而仍保持最小的可能。本题的最小生成树不仅总权值为 15，而且边集唯一。

6.3 图形化分析

1. **Kruskal 图**：绿色边表示已经接受的边，蓝色表示当前接受边，红色虚线表示会成环而被舍弃的边。图中清楚显示算法先形成若干小分量，再逐步合并。
2. **Prim 图**：绿色顶点和边表示当前生成树，橙色边表示当前割上的候选边，蓝色边表示本轮选择。算法始终维持一个连通的树结构，并在每轮增加一个顶点。
3. **结果对比图**：圆形编号表示选边次序。本题两种算法的规范化边序列恰好相同，但这不是算法的一般保证，而是图结构、起点和同权边规则共同作用的结果。

6.4 结果不同时应如何解释

1. **仅端点方向不同**： $V_3-V_4(2)$ 与 $V_4-V_3(2)$ 在无向图中完全等价，不属于结果差异。
2. **选边顺序不同但最终边集相同**：Kruskal 按全局边序处理，Prim 受起点和当前访问集合影响，因此过程不同是正常的。
3. **最终边集不同但总权值相同**：在存在多棵最小生成树的图中，这是正常现象。只要两者均为合法生成树且达到最小总权值，就都是正确答案。
4. **本题最终边集不同**：由于前文已经证明本题 MST 唯一，这种情况不是“多解”，应检查边权录入、无向边建图、并查集判环、Prim 访问标记或候选边失效判断。
5. **总权值不同**：至少有一个实现或输入存在错误；应首先比较逐步选边日志，定位第一次出现差异的步骤。

6.5 复杂度分析

设顶点数为 V ，边数为 E 。

- Kruskal 的主要开销是边排序，为 $O(E \log E)$ ；并查集操作总计约为 $O(E\alpha(V))$ 。边数组、结果和并查集空间总计为 $O(E + V)$ 。
- 本实验的 Prim 使用邻接表与二叉堆。每条无向边最多以两个方向进入邻接表，候选边的压入和弹出带来 $O(E \log E)$ 时间；常用简化表示为 $O(E \log V)$ 。空间复杂度为 $O(E + V)$ 。

对本题 $V = 6, E = 10$ ，两种算法都能快速完成。若图很稀疏且输入直接给出边集，Kruskal 实现通常更简洁；若问题强调从某个已连接区域持续扩展，Prim 的过程更直观。

7 完整程序代码

前文已经按功能解释了关键代码。以下给出本实验实际编译运行的完整 C++17 源码，便于核对头文件、辅助函数、输出格式、异常处理和整个调用关系。

代码清单 9: Kruskal 与 Prim 完整 C++17 实现

```
1 #include <algorithm>
2 #include <iomanip>
3 #include <iostream>
4 #include <numeric>
5 #include <queue>
6 #include <stdexcept>
7 #include <tuple>
8 #include <vector>
9
10 using namespace std;
11
```



```
12 struct Edge {
13     int u;
14     int v;
15     int weight;
16 };
17
18 // Disjoint-set union supports cycle detection in Kruskal.
19 class DisjointSet {
20 public:
21     explicit DisjointSet(int n) : parent(n + 1), rank_value(n + 1, 0) {
22         iota(parent.begin(), parent.end(), 0);
23     }
24
25     int find(int x) {
26         if (parent[x] != x) {
27             parent[x] = find(parent[x]);
28         }
29         return parent[x];
30     }
31
32     bool unite(int a, int b) {
33         int root_a = find(a);
34         int root_b = find(b);
35         if (root_a == root_b) {
36             return false;
37         }
38
39         if (rank_value[root_a] < rank_value[root_b]) {
40             swap(root_a, root_b);
41         }
42         parent[root_b] = root_a;
43         if (rank_value[root_a] == rank_value[root_b]) {
44             ++rank_value[root_a];
45         }
46         return true;
47     }
48
49 private:
50     vector<int> parent;
51     vector<int> rank_value;
52 };
53
54 tuple<int, int, int> normalizedKey(const Edge& edge) {
55     return {edge.weight, min(edge.u, edge.v), max(edge.u, edge.v)};
56 }
57
58 bool edgeLess(const Edge& a, const Edge& b) {
59     return normalizedKey(a) < normalizedKey(b);
60 }
61
```

```

62 vector<Edge> kruskal(int vertex_count, vector<Edge> edges) {
63     // Equal-weight edges are ordered by their normalized endpoints.
64     sort(edges.begin(), edges.end(), edgeLess);
65
66     DisjointSet dsu(vertex_count);
67     vector<Edge> mst;
68
69     for (const Edge& edge : edges) {
70         if (dsu.unite(edge.u, edge.v)) {
71             mst.push_back(edge);
72             if (static_cast<int>(mst.size()) == vertex_count - 1) {
73                 break;
74             }
75         }
76     }
77
78     if (static_cast<int>(mst.size()) != vertex_count - 1) {
79         throw runtime_error("The graph is disconnected; no spanning tree exists.");
80     }
81     return mst;
82 }
83
84 vector<Edge> prim(int vertex_count, const vector<Edge>& edges, int start) {
85     // Build an undirected adjacency list.
86     vector<vector<pair<int, int>>> graph(vertex_count + 1);
87     for (const Edge& edge : edges) {
88         graph[edge.u].push_back({edge.v, edge.weight});
89         graph[edge.v].push_back({edge.u, edge.weight});
90     }
91
92     // weight, smaller endpoint, larger endpoint, from, to
93     using Candidate = tuple<int, int, int, int, int>;
94     priority_queue<Candidate, vector<Candidate>, greater<Candidate>> candidates;
95     vector<bool> visited(vertex_count + 1, false);
96     vector<Edge> mst;
97
98     auto addCandidates = [&](int vertex) {
99         visited[vertex] = true;
100         for (const auto& [next, weight] : graph[vertex]) {
101             if (!visited[next]) {
102                 candidates.push(
103                     {weight, min(vertex, next), max(vertex, next), vertex, next});
104             }
105         }
106     };
107
108     addCandidates(start);
109     while (!candidates.empty() &&
110         static_cast<int>(mst.size()) < vertex_count - 1) {
111         auto [weight, smaller, larger, from, to] = candidates.top();

```

```
112     candidates.pop();
113     (void)smaller;
114     (void)larger;
115
116     if (visited[to]) {
117         // This candidate became stale after both endpoints were visited.
118         continue;
119     }
120
121     mst.push_back({from, to, weight});
122     addCandidates(to);
123 }
124
125 if (static_cast<int>(mst.size()) != vertex_count - 1) {
126     throw runtime_error("The graph is disconnected; no spanning tree exists.");
127 }
128 return mst;
129 }
130
131 int totalWeight(const vector<Edge>& mst) {
132     int total = 0;
133     for (const Edge& edge : mst) {
134         total += edge.weight;
135     }
136     return total;
137 }
138
139 bool isSpanningTree(int vertex_count, const vector<Edge>& edges) {
140     // A spanning tree on n vertices must contain exactly n - 1 edges.
141     if (static_cast<int>(edges.size()) != vertex_count - 1) {
142         return false;
143     }
144
145     DisjointSet dsu(vertex_count);
146     for (const Edge& edge : edges) {
147         if (!dsu.unite(edge.u, edge.v)) {
148             return false;
149         }
150     }
151
152     const int root = dsu.find(1);
153     for (int vertex = 2; vertex <= vertex_count; ++vertex) {
154         if (dsu.find(vertex) != root) {
155             return false;
156         }
157     }
158     return true;
159 }
160
161 vector<tuple<int, int, int>> normalizedEdgeSet(const vector<Edge>& edges) {
```

```

162     vector<tuple<int, int, int>> result;
163     result.reserve(edges.size());
164     for (const Edge& edge : edges) {
165         result.push_back(normalizedKey(edge));
166     }
167     sort(result.begin(), result.end());
168     return result;
169 }
170
171 void printResult(const string& algorithm, const vector<Edge>& mst) {
172     cout << algorithm << '\n';
173     cout << "Step Selected edge Weight\n";
174
175     for (size_t i = 0; i < mst.size(); ++i) {
176         const Edge& edge = mst[i];
177         cout << setw(4) << i + 1 << " V" << edge.u << "-V" << edge.v
178             << setw(12) << edge.weight << '\n';
179     }
180
181     cout << "Edge count: " << mst.size() << '\n';
182     cout << "Total weight: " << totalWeight(mst) << "\n\n";
183 }
184
185 int main() {
186     const int vertex_count = 6;
187     const vector<Edge> edges = {
188         {1, 2, 4}, {1, 3, 3}, {1, 4, 1}, {2, 4, 5}, {2, 5, 7},
189         {3, 4, 2}, {3, 6, 3}, {4, 5, 5}, {4, 6, 4}, {5, 6, 6},
190     };
191
192     try {
193         const vector<Edge> kruskal_mst = kruskal(vertex_count, edges);
194         const vector<Edge> prim_mst = prim(vertex_count, edges, 1);
195
196         printResult("Kruskal algorithm", kruskal_mst);
197         printResult("Prim algorithm (start from V1)", prim_mst);
198
199         const bool same_weight = totalWeight(kruskal_mst) == totalWeight(prim_mst);
200         const bool same_edges =
201             normalizedEdgeSet(kruskal_mst) == normalizedEdgeSet(prim_mst);
202         const bool kruskal_valid =
203             isSpanningTree(vertex_count, kruskal_mst);
204         const bool prim_valid = isSpanningTree(vertex_count, prim_mst);
205
206         cout << "Verification\n";
207         cout << "Kruskal is a spanning tree: "
208             << (kruskal_valid ? "YES" : "NO") << '\n';
209         cout << "Prim is a spanning tree: " << (prim_valid ? "YES" : "NO")
210             << '\n';
211         cout << "Same total weight: " << (same_weight ? "YES" : "NO") << '\n';

```

```
212     cout << "Same normalized edge set: " << (same_edges ? "YES" : "NO")
213         << '\n';
214     cout << "Verification passed: "
215         << (kruskal_valid && prim_valid && same_weight ? "YES" : "NO")
216         << '\n';
217
218     // Different edge sets can both be correct when an MST is not unique.
219     return kruskal_valid && prim_valid && same_weight ? 0 : 1;
220 } catch (const exception& error) {
221     cerr << "Error: " << error.what() << '\n';
222     return 1;
223 }
224 }
```

8 实验结论与反思

本实验使用两种经典贪心算法求得相同的最小生成树：

$$T = \{V_1V_4, V_3V_4, V_3V_6, V_1V_2, V_4V_5\}, \quad w(T) = 15.$$

Kruskal 的核心是“按边从小到大尝试，并使用并查集避免成环”；Prim 的核心是“维护当前树与外部顶点之间的割，每次选择最轻跨边”。二者虽然观察问题的角度不同，但都依赖割性质保证每次贪心选择是安全的。

实验中最容易混淆的地方是把“执行过程相同”误当作“算法正确”的必要条件。实际上，起点、同权边顺序以及 MST 是否唯一都会影响输出边的次序甚至最终边集。因此程序验证应首先检查生成树合法性和总权值，再将边集一致性作为辅助信息。本题可由五个单顶点割上的唯一最轻边证明 MST 唯一，所以两种正确实现最终应得到同一规范化边集，这也为程序调试提供了更强的检查依据。